Southern New Hampshire University

CS 255 System Design Document

DriverPass | Robert Wilburn | Southern New Hampshire University

UML Diagrams

UML Use Case Diagram

The use case diagram below shows the complete DriverPass system inside a single system boundary. Seven actors interact with it. Five are people, the Customer, the Secretary, the Driver, the IT Officer, and the Owner, and two are outside systems, the DMV and a payment processor, both of which exchange data with DriverPass without being part of it.

Shared behavior is modeled with «include» relationships so it is defined once rather than repeated. Logging in includes validating credentials, scheduling a lesson includes assigning a driver and car, and modifying or canceling a reservation includes writing an activity log entry, which is what allows the owner to trace every booking change back to the user responsible. Resetting a forgotten password is drawn as an «extend» because it occurs only in the exceptional case where authentication fails.

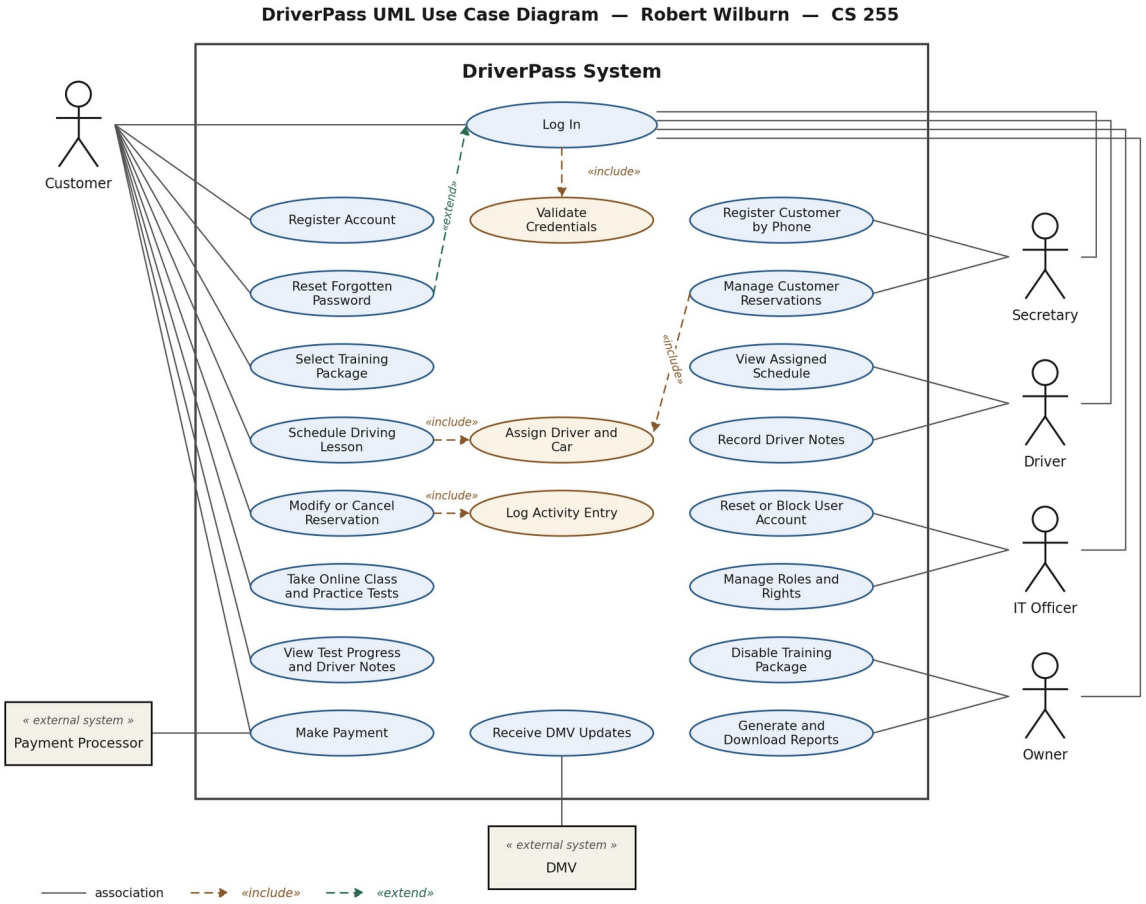


Figure 1. UML use case diagram for the DriverPass system.

UML Activity Diagrams

Two use cases were selected for decomposition. Scheduling a driving lesson was chosen because it is the central transaction of the business and touches customers, drivers, cars, package hours, and the audit log. Registering an account and purchasing a training package was chosen because it is the entry point for every customer and is the only flow that involves the external payment processor.

Activity Diagram 1: Schedule a Driving Lesson

This flow validates credentials, confirms the customer still has hours remaining under their package, and checks driver and car availability before anything is committed. A request that cannot be filled returns alternative times and loops back to date selection rather than failing outright. Once a slot is confirmed, the system assigns the driver and car, confirms pickup and drop-off locations, saves the reservation together with an audit entry, and sends a confirmation to the customer.

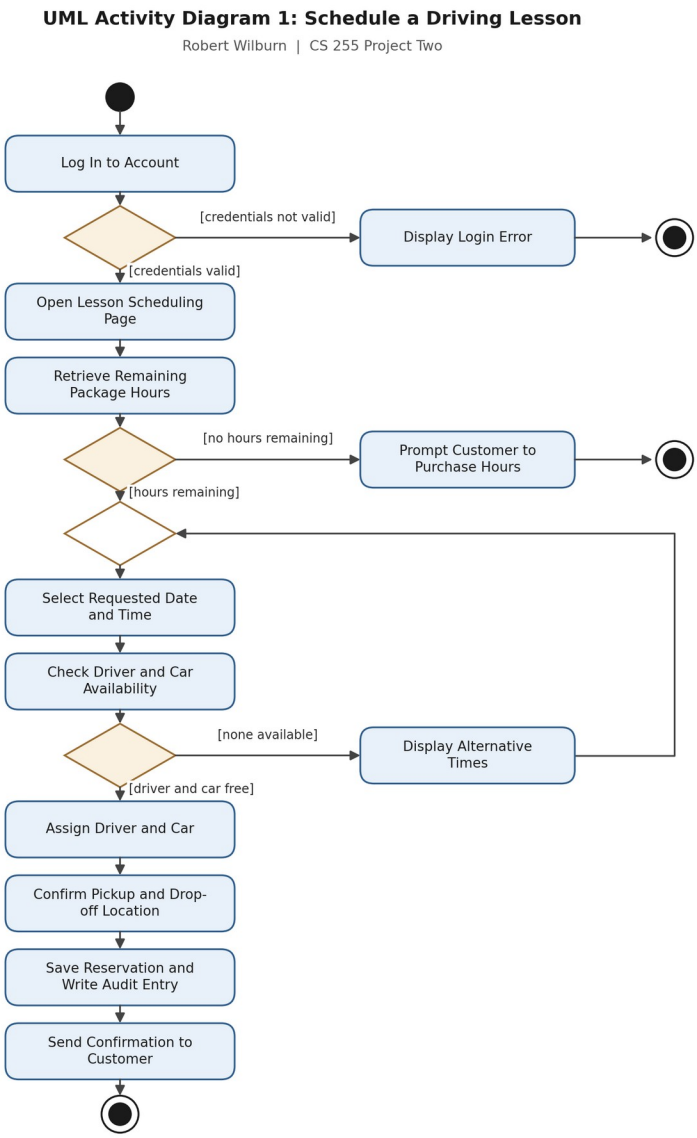


Figure 2. UML activity diagram for the Schedule a Driving Lesson use case.

Activity Diagram 2: Register Account and Purchase Training Package

This flow captures the registration details the client described in the interview, including name, contact information, pickup location, and a drop-off location that defaults to the pickup address unless the customer specifies otherwise. Field validation loops back rather than discarding the entry. Payment is submitted to an external processor and the account is created only after the charge is approved, so no customer is granted package access without a completed payment.

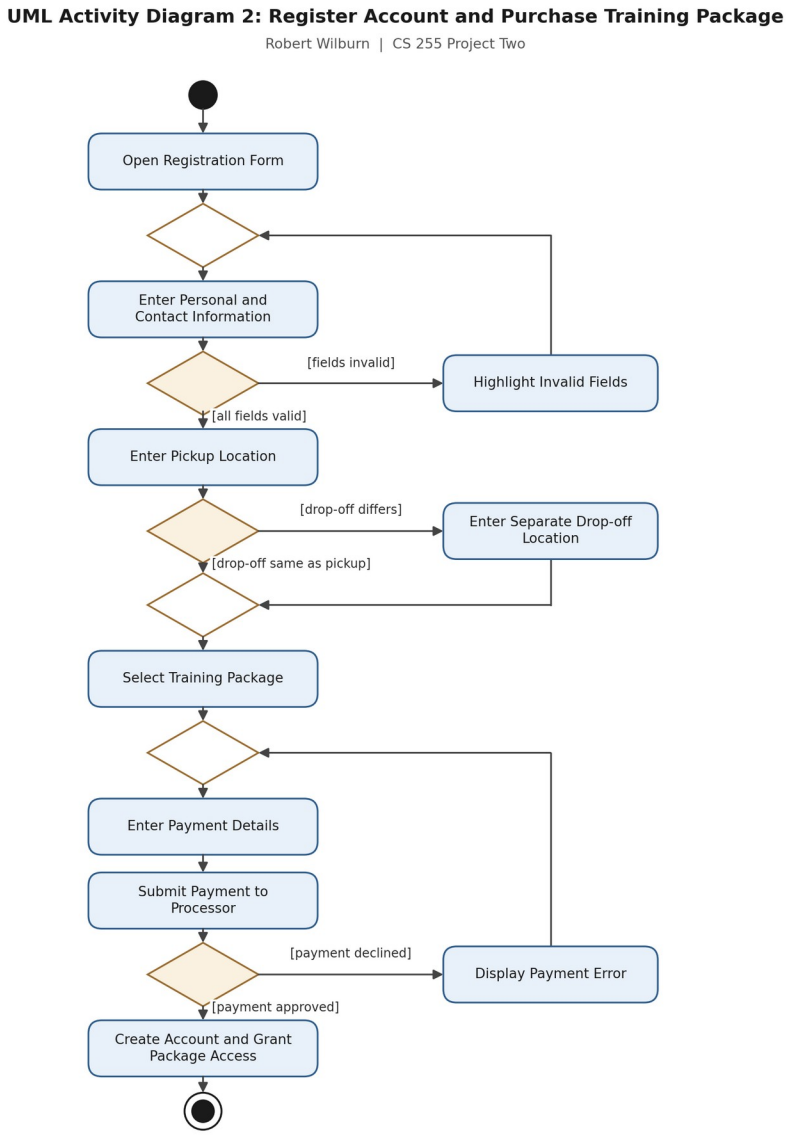


Figure 3. UML activity diagram for the Register Account and Purchase Training Package use case.

UML Sequence Diagram

The sequence diagram below models the Schedule a Driving Lesson use case as an exchange of messages over time. It shows five participants: the Customer, the Web Interface that the customer

works in, a Reservation Manager that holds the scheduling logic, the Database, and a Notification Service.

Separating the Reservation Manager from the Web Interface matters to the design, because the secretary books lessons through a different screen than the customer does while both must enforce the same rules. Placing those rules in one component rather than in the interface keeps the two paths consistent. An alt fragment covers the three possible outcomes: a successful booking, a request where no driver or car is free, and a request from a customer with no hours remaining. Solid arrows are calls and dashed arrows are returns.

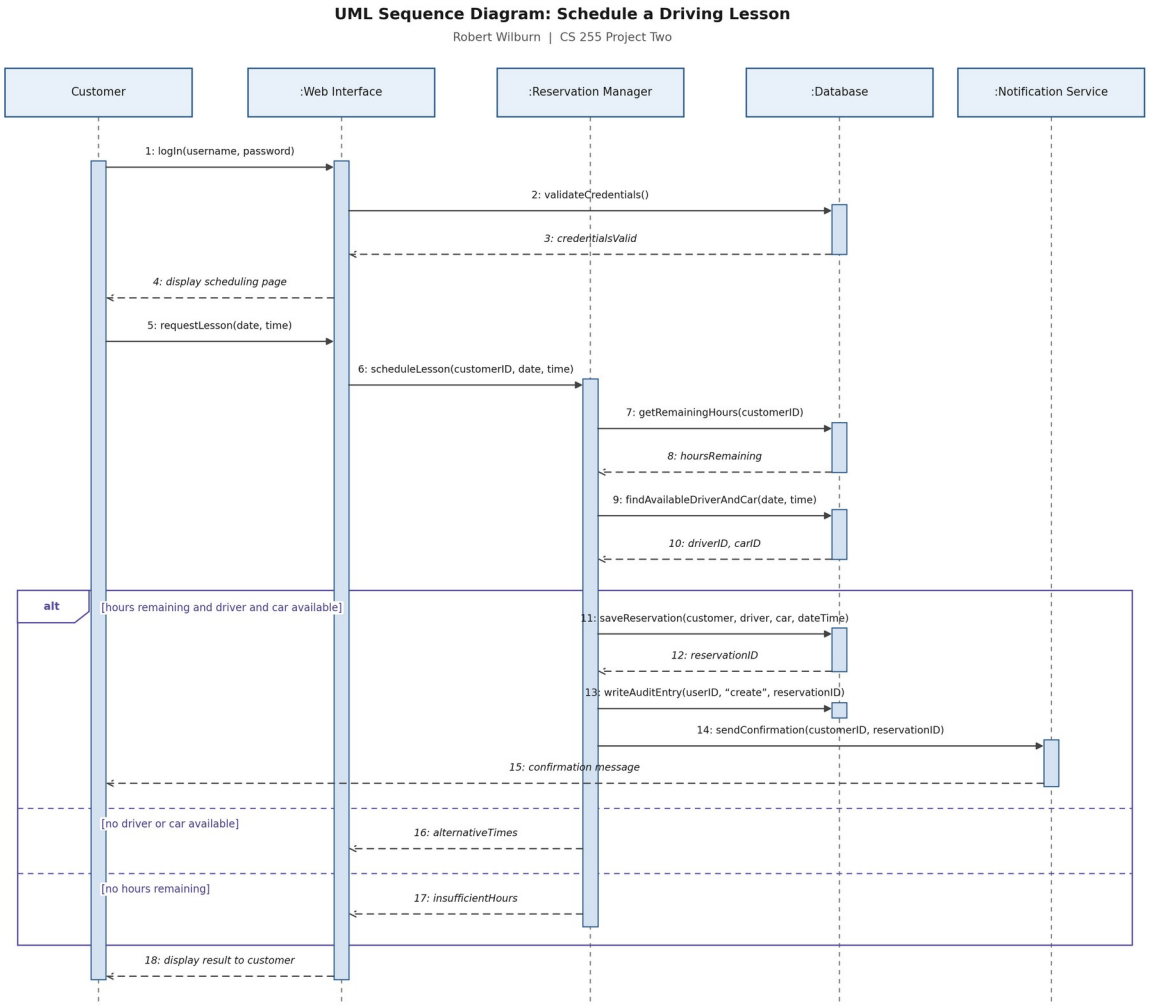


Figure 4. UML sequence diagram for the Schedule a Driving Lesson use case.

UML Class Diagram

The class diagram records the structure of the system. A general User class holds the credentials and account state shared by everyone who signs in, and Customer and Employee inherit from it, with Driver inheriting in turn from Employee. This is what lets the system support five kinds of users without

repeating login behavior five times, and it means a new type of employee can be added later as a subclass rather than by rewriting the access logic.

Reservation sits at the center of the model because it ties a customer to a driver, a car, a date, and a two-hour block. Car carries its own availability so that no vehicle is double booked, DriverNote records the lesson feedback the client asked to display, and AuditEntry captures who performed each action and when. Package holds an isActive flag, which is how the owner disables a package so no further customers can register for it without removing it from customers who already bought it. Payment card data is deliberately absent from the model, since those details are handled by the external payment processor rather than stored by DriverPass.

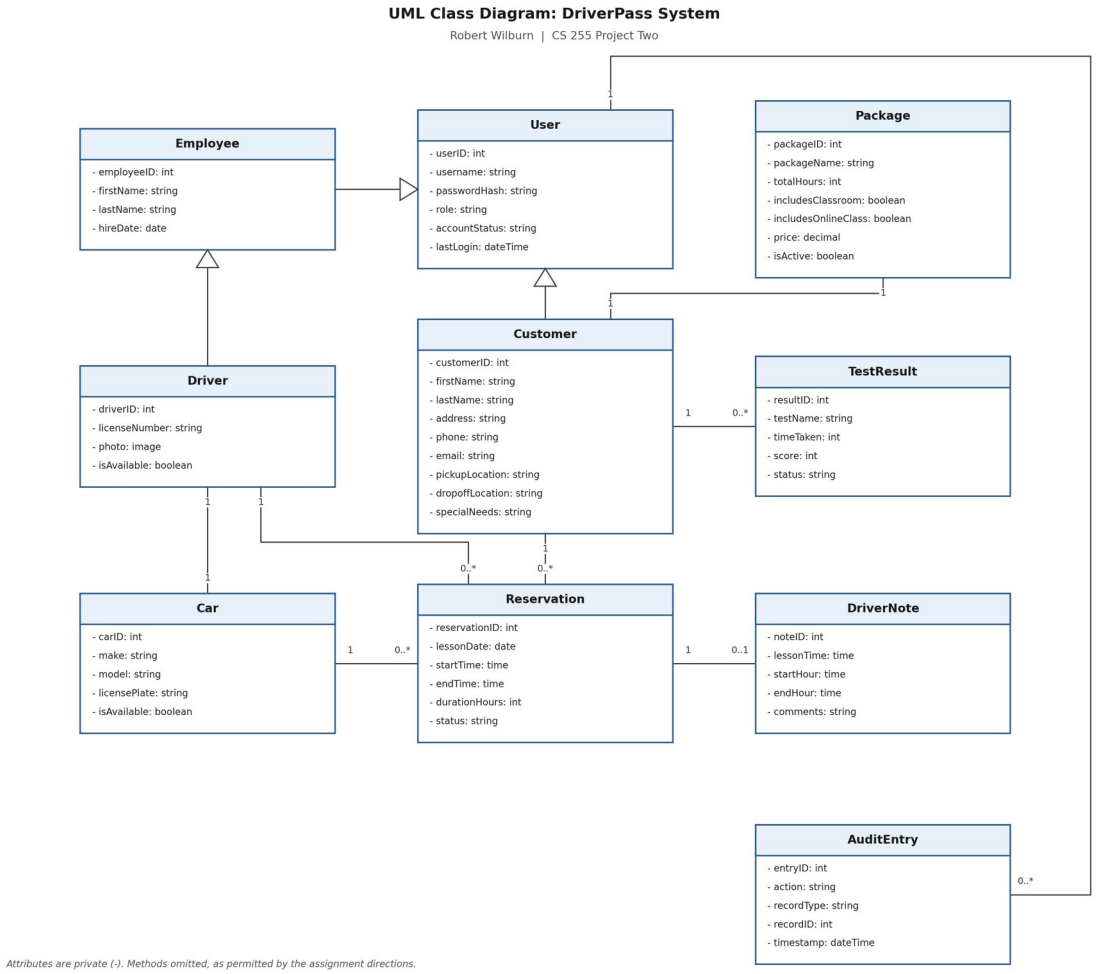


Figure 5. UML class diagram for the DriverPass system.

Technical Requirements

The technical requirements below follow from the diagrams above and from the functional and nonfunctional requirements documented in the business requirements document.

Infrastructure

- The system shall be deployed as a cloud-hosted web application rather than on servers located at the DriverPass office. The IT officer stated that DriverPass does not want to manage backup and security itself, so those responsibilities belong to the hosting provider.
- The hosting environment shall provide automated daily backups, encryption of stored data, and a documented recovery time objective, since grades, reservations, and customer records cannot be reconstructed if lost.
- The environment shall support horizontal scaling so that additional capacity can be added as the customer base grows, rather than requiring the system to be rebuilt once demand exceeds the original sizing.
- The system shall maintain separate development, testing, and production environments so that changes can be validated before customers encounter them.
- Availability shall meet a target of 99.9 percent measured monthly, because customers book lessons through the website outside of office hours.

Hardware

- Application and database servers shall be provisioned from the cloud provider rather than purchased, which keeps initial costs down for a new company and allows capacity to be adjusted.
- No specialized hardware is required at the client side. Customers and staff need only a computer, tablet, or smartphone with a current web browser and an internet connection.
- The DriverPass office requires standard workstations for the secretary and the owner and a printer capable of producing the activity report the owner requested.
- Drivers require a mobile device with a browser and a data connection so they can view their assigned schedule and enter lesson notes from the vehicle.
- Each of the ten vehicles is a tracked resource within the system, so the fleet size sets the upper bound on how many lessons can be booked in any two-hour block.

Software

- The front end shall be built with standard web technologies, HTML, CSS, and JavaScript, using a responsive framework so that the same pages work on desktop, tablet, and phone screens without separate native applications.
- The back end shall be implemented in an object-oriented language such as Java, C#, or Python, which maps directly to the classes and relationships defined in the class diagram above.
- A relational database management system such as PostgreSQL, MySQL, or SQL Server shall store the entities in the class diagram and shall enforce referential integrity so that a reservation cannot exist without a valid customer, driver, and car.
- The system shall support the current released versions of Chrome, Firefox, Safari, and Edge. No browser plug-ins shall be required.

- Reports shall be exportable in a format that opens in Excel, which the owner specifically requested so he can work with data offline.

Tools and Integrations

- An established payment processing service such as Square or Stripe shall handle credit card authorization. DriverPass shall store only the token and transaction reference returned by that service, never the card number, expiration date, or security code.
- An interface to the DMV shall receive rule, policy, and sample question updates. Because the exchange format is controlled by the DMV rather than by DriverPass, this integration shall be built as a separate component that can be changed without altering the rest of the system.
- An email or messaging service shall deliver reservation confirmations, password reset links, and the notification sent when a DMV update arrives.
- Version control and an automated build and deployment pipeline shall be used so that releases are repeatable and can be rolled back.
- Automated testing tools shall cover unit, integration, and user acceptance testing during the ten-day test window allocated in the project schedule.

Security

- All traffic between client and server shall be encrypted in transit using TLS, and customer data shall be encrypted at rest.
- Passwords shall be stored as salted hashes rather than recoverable text, and staff and administrative accounts shall additionally require multifactor authentication.
- Role-based access control shall be enforced in the business logic layer rather than in the browser, so that an account in one role cannot reach a function reserved for another.
- Package entitlement shall be enforced separately from employee roles, since what a paying customer may open depends on which package they purchased rather than on a job function.
- Repeated failed login attempts shall lock the account automatically and notify the IT officer, and the audit log shall be readable but not editable so that it can establish responsibility when something goes wrong.